

Relatório Técnico Completo

Sistema de Agendamento de Atendimentos — agenda Saúde

Equipe agenda Saúde — PI2

Documentação: estruturas, código, funcionalidades, API, construção e referências.

Versão do sistema: 1.0.9 · Data: 08/09/2026 · Status: operacional 50 testes RLS ativo

Sumário

- 1. Visão geral
- 2. Arquitetura e stack tecnológica
- 3. Estrutura de pastas
- 4. Modelagem de dados (Supabase/PostgreSQL)
- 5. API REST — endpoints e contratos
- 6. Descrição das funções por camada
- 7. Funcionalidades (features)
- 8. Construção (build, deploy e testes)
- 9. Segurança
- 10. Pendências e evoluções
- 11. Referências bibliográficas

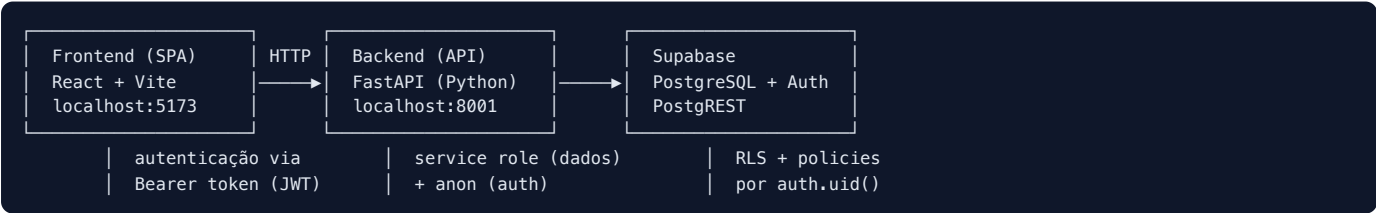
1. Visão geral

O **agenda Saúde** é um sistema web de agendamento de atendimentos voltado a unidades de saúde, que permite cadastrar pacientes e agentes comunitários, montar uma agenda (planner diário/semanal), criar, reagendar, cancelar e acompanhar consultas, com trilha de auditoria e notificação por e-mail ao paciente. É um projeto integrador (PI2) desenvolvido em camadas, com backend em Python/FastAPI, frontend em React/TypeScript e persistência + autenticação no Supabase (PostgreSQL).

Objetivo: organizar a rotina de um operador/recepcionista de uma unidade de saúde, reduzindo faltas (no-show) e melhorando o aproveitamento da agenda.

2. Arquitetura e stack tecnológica

2.1. Visão arquitetural



2.2. Stack

Camada	Tecnologias	Papel
Frontend	React 18, TypeScript (strict), Vite, Tailwind CSS v4, shadcn/ui (Radix), Recharts, date-fns, lucide-react, sonner, react-router	SPA; UI em componentes; contrato de API em services/
Backend	Python 3.14, FastAPI, Pydantic v2, supabase-py, httpx, redis	API REST em camadas controller → service → repository; validação e regras
Dados/Auth	Supabase (PostgreSQL 15, Auth JWT, PostgREST), RLS	Persistência, autenticação, autorização por linha
Integrações	Brevo (e-mail transacional), Redis (rate limit, opcional)	Notificações e controle de abuso
Deploy	Vercel (frontend), Render (backend), GitHub	Hospedagem e versionamento

2.3. Padrões e princípios

- Arquitetura em camadas:** `controllers` (HTTP) → `services` (regras de negócio) → `repositories` (acesso a dados) → Supabase.
- Validação única:** Pydantic nos schemas; validações de negócio concentradas nos services (ex.: horário cheio e UTC).

- **Handler global de erros:** respostas padronizadas `{success, message, errors}` via `ValidationException`.
- **UTC de ponta a ponta:** datas em `timestampz`; conversão local (`America/Sao_Paulo`) apenas na exibição/e-mails.
- **Cliente HTTP único:** `apiFetch` no frontend com refresh automático de sessão.

3. Estrutura de pastas

3.1. Backend

```
backend/
├─ main.py           # App FastAPI + CORS + handler global de erros
├─ config/
│  └─ database.py    # Clientes supabase (anon) e db (service role)
│  └─ dependencies.py # get_current_user (HTTPBearer → JWT)
├─ controllers/      # Rotas HTTP (sem try/except)
│  └─ auth_controller.py # register, login, refresh, forgot/reset
│  └─ agendamento_controller.py
│  └─ paciente_controller.py
│  └─ agentecomunitario_controller.py
│  └─ statusagendamento_controller.py
│  └─ tiposusuarios_controller.py
│  └─ usuario_controller.py
├─ services/         # Regras de negócio
│  └─ auth_service.py # Register, login, refresh, forgot/reset
│  └─ agendamento_service.py # Criar/atualizar/cancelar + auditoria + notificação
│  └─ paciente_service.py
│  └─ agentecomunitario_service.py
│  └─ statusagendamento_service.py
│  └─ tiposusuarios_service.py
│  └─ usuario_service.py # get_me, update, delete (Auth)
│  └─ notificacao_service.py # E-mail via Brevo
│  └─ base_service.py # now() UTC
├─ repositories/     # Acesso a dados (via db/service role)
│  └─ base_repository.py
│  └─ agendamento_repository.py
│  └─ agendamento_auditoria_repository.py
│  └─ paciente_repository.py
│  └─ agentecomunitario_repository.py
│  └─ statusagendamento_repository.py
│  └─ tiposusuarios_repository.py
│  └─ usuario_repository.py
├─ schemas/          # Pydantic (Create/Update/Response)
├─ utils/             # rate_limit, response, validators
├─ exceptions/        # ValidationException
├─ sql/               # migracoes_soft_delete, auditoria, rls
├─ scripts/           # verificar_migracoes, testar_brevo
└─ tests/             # pytest (50 casos)
```

3.2. Frontend

```
frontend/src/
├─ main.tsx          # bootstrap React + index.css
├─ app/
│  └─ App.tsx         # RouterProvider + Toaster
│  └─ routes.tsx      # Configuração de rotas
│  └─ pages/          # 14 telas
│  └─ components/     # DashboardLayout, AttendanceStats, GuiaOverlay, upvisual/
│     └─ ui/          # shadcn/ui (50+ primitivos Radix)
│  └─ services/       # apiClient, agendamentoService, pacienteService,
│                     # agenteService, session, upVisualData
│  └─ types/          # agendamento.ts (Agendamento, RegistroAuditoria)
│  └─ constants/      # horarios.ts
│  └─ styles/         # index.css, tailwind.css, theme.css
└─ assets/           # login-bg.jpeg
```

4. Modelagem de dados (Supabase/PostgreSQL)

4.1. Tabelas

Tabela	Chave	Campos principais	RLS
--------	-------	-------------------	-----

<code>usuarios</code>	<code>usuario_id</code> (uuid = auth.uid)	nome, email, tipousuario_id, criado_em, atualizado_em	✔ policies por auth.uid()
<code>tiposusuarios</code>	<code>tipousuario_id</code>	nome	✔ acesso autenticado
<code>agentescomunitarios</code>	<code>agentecomunitario_id</code>	nome, criado_em	✔ acesso autenticado
<code>pacientes</code>	<code>paciente_id</code>	nome, numero_sus, email (obrigatório), telefone, data_nascimento, agentecomunitario_id (FK), status, observacoes	✔ acesso autenticado
<code>statusagendamento</code>	<code>statusagendamento_id</code>	nome	✔ acesso autenticado
<code>agendamentos</code>	<code>agendamento_id</code>	usuario_id (FK), paciente_id (FK), statusagendamento_id (FK), data_hora_inicio/fim (timestampz), observacoes, cancelado (soft delete), criado_em, atualizado_em	✔ acesso autenticado
<code>agendamentos_auditoria</code>	<code>auditoria_id</code>	agendamento_id, usuario_id, acao (criado/atualizado/cancelado), dados (jsonb), criado_em	✔ só service role

4.2. Migrações

Arquivo	Conteúdo
<code>sql/migracao_soft_delete.sql</code>	Coluna <code>cancelado</code> + horários <code>timestampz</code>
<code>sql/migracao_auditoria.sql</code>	Tabela <code>agendamentos_auditoria</code> + índices
<code>sql/migracao_rls.sql</code>	<code>ENABLE ROW LEVEL SECURITY</code> em 7 tabelas + policies

5. API REST — endpoints e contratos

5.1. Autenticação (`/auth`)

Método	Rota	Corpo (entrada)	Resposta
POST	<code>/auth/register</code>	{nome, email, password, tipousuario_id}	201 {user:{id,email,nome}} · rate limit
POST	<code>/auth/login</code>	{email, password}	200 {access_token, refresh_token, user}
POST	<code>/auth/refresh</code>	{refresh_token}	200 {access_token, refresh_token}
POST	<code>/auth/forgot-password</code>	{email}	200 {message} (sempre sucesso)
POST	<code>/auth/reset-password</code>	{access_token, refresh_token, password}	200 {message}

5.2. Agendamentos (`/agendamentos`) — autenticado

Método	Rota	Descrição
GET	<code>/agendamentos/</code>	Listar (limit/offset) com joins de paciente/status/usuário
GET	<code>/agendamentos/{id}</code>	Buscar por id
POST	<code>/agendamentos/</code>	Criar (valida horário cheio/futuro/conflicto; grava auditoria; notifica paciente via Brevo)
PUT	<code>/agendamentos/{id}</code>	Atualizar/reagendar (evita conflito com o próprio id)
DELETE	<code>/agendamentos/{id}</code>	Cancelar (soft delete: <code>cancelado=true</code>) + auditoria
GET	<code>/agendamentos/{id}/auditoria</code>	Trilha de auditoria com nome do autor

5.3. Pacientes / Agentes / Status / Tipos / Usuários

Recurso	Endpoints	Notas
<code>/pacientes</code>	GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}	Email obrigatório (422); telefone normalizado; FK de agente
<code>/agentescomunitarios</code>	GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}	Cadastro e renomeio
<code>/statusagendamento</code>	GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}	Status de agendamento
<code>/tiposusuarios</code>	GET / (público), GET /{id}, POST /, PUT /{id}, DELETE /{id}	GET / é público (usado na tela de registro)
<code>/usuarios</code>	GET /me, GET /{user_id}, PUT /{user_id}, DELETE /{user_id}	Somente o próprio usuário; DELETE também remove do Supabase Auth (service role) e tem guard de FK

5.4. Utilitários

GET / (status da API) e GET /health (healthcheck).

6. Descrição das funções por camada

6.1. Backend — services (principais)

Classe/Método	Responsabilidade
AuthService.register()	Cria usuário no Supabase Auth, insere perfil em <code>usuarios</code> , valida senha mínima e tipo de usuário
AuthService.login()	Autentica no Auth, busca perfil em <code>usuarios</code> , retorna access/refresh token e dados do usuário
AuthService.refresh()	Renova a sessão via <code>refresh_session</code>
AuthService.forgot_password()	Dispara e-mail de recuperação (sempre responde sucesso)
AuthService.reset_password()	Valida tokens do link e troca a senha em cliente isolado
AgendamentoService.criar()	Valida FKs, normaliza datetime (UTC), valida horário cheio/futuro, checa conflito, cria, registra auditoria e notifica paciente
AgendamentoService.atualizar()	Reagendamento: valida novo horário, ignora o próprio id no conflito, registra auditoria
AgendamentoService.deletar()	Soft delete (<code>cancelado=true</code>) + auditoria "cancelado" com autor
AgendamentoService.listar_auditoria()	Retorna trilha com nome/e-mail do autor (via <code>buscar_nomes</code>)
NotificacaoService.confirmar_agendamento()	Envia e-mail transacional via Brevo (horário em America/Sao_Paulo); falha não interrompe o fluxo
UsuarioService.delete()	Remove usuário do Supabase Auth (service role) + linha da tabela; guard de agendamentos vinculados (400)
RateLimit (RedisRateLimiter/MemoryRateLimiter)	Controle de 5 req/min por IP em auth; Redis com fallback memória

6.2. Frontend — services e helpers

Função	Responsabilidade
apiFetch(path, options)	Injeta token, em 401 renova a sessão (single-flight) e repete a requisição
refreshSession()	Chama <code>/auth/refresh</code> e atualiza tokens no localStorage
fetchAgendamentos / fetchAgendamentoById / create / update / cancelAgendamento	Contrato de API de agendamentos (normaliza snake_case → camelCase)
fetchAgendamentoAuditoria(id)	Busca a trilha de auditoria
fetchPacientes() / fetchAgentes() / atualizarAgente()	Contrato das listagens
handleUnauthorized(status)	Limpa sessão e redireciona ao login em 401
upVisualData.*	Dados mock determinísticos + engine de insights do painel Up Visual (provisório)

6.3. Frontend — telas (pages)

Tela	Função
LoginPage	Autenticação com fundo, link de recuperação e avisos
RegisterPage	Cadastro (lista tipos de usuário via rota pública)
ForgotPasswordPage / ResetPasswordPage	Recuperação de senha (lê tokens do hash da URL)
PlannerPage	Planner diário/semanal; alteração de status e reagendamento
AppointmentsPage	Listagem/filtros de consultas + ações (reagendar, cancelar, histórico)
NewAppointmentPage	Criação de agendamento
ReschedulePage	Reagendamento
PacientesPage / NewPatientPage / EditPatientPage	Listar/criar/editar pacientes (e-mail obrigatório com validação)
AgentesComunitariosPage / NewAgenteComunitarioPage	Listar/criar agentes e renomear
UpVisualPage	Dashboard analítico (provisório; calendário de ocupação, KPIs, gráficos, insights)

7. Funcionalidades (features)

- **Autenticação completa:** registro, login, refresh automático de sessão, recuperação de senha.
- **Gestão de agenda:** planner diário/semanal, criação, reagendamento e cancelamento (soft delete que libera o horário).
- **Cadastros:** pacientes (e-mail obrigatório) e agentes comunitários, com listagens e filtros.
- **Trilha de auditoria:** quem criou/alterou/cancelou e quando, visível em "Histórico".
- **Notificação por e-mail (Brevo):** confirmação automática de agendamento ao paciente.
- **Segurança:** RLS no banco, rotas protegidas por JWT, rate limit (Redis/memória), exclusão de conta também no Auth.
- **UX:** menu de navegação, tema escuro e guia passo a passo (habilitáveis por engrenagem), fundo na tela de login.
- **Painel analítico (provisório):** calendário de ocupação (heatmap), KPIs, gráficos e insights com dados mock.

8. Construção (build, deploy e testes)

8.1. Local

```
# Backend
cd backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env          # SUPABASE_URL, SUPABASE_KEY, SUPABASE_SERVICE_ROLE_KEY, FRONTEND_URL, BREVO_*
uvicorn src.main:app --reload  # http://localhost:8001

# Frontend
cd frontend
npm install
echo "VITE_API_URL=http://localhost:8001" > .env.local
npm run dev                  # http://localhost:5173
```

8.2. Testes

```
cd backend
python -m pytest tests -q      # 50 passed (fakes, sem rede)
cd frontend
npx tsc --noEmit               # typecheck
npm run build                  # tsc + vite build
```

8.3. Scripts utilitários

- `scripts/verificar_migracoes.py` — valida se as migrações SQL foram aplicadas.
- `scripts/testar_brevo.py` — envia e-mail de teste via Brevo.

8.4. Deploy

Componente	Hospedagem	Passo
Frontend	Vercel	Importar repo; <code>vercel.json</code> cobre rotas SPA; <code>VITE_API_URL</code> = URL do backend
Backend	Render	Service apontando para <code>backend/</code> ; <code>uvicorn src.main:app --host 0.0.0.0 --port \$PORT</code> ; envs de produção
Banco	Supabase	Migrações aplicadas; RLS habilitado; redirects de recuperação de senha

9. Segurança

- **Autenticação:** JWT (Supabase Auth) exigido em todas as rotas de dados (HTTPBearer, 401).
- **Autorização por linha:** RLS habilitado; polices por `auth.uid()` em `usuarios` e acesso autenticado nas demais.
- **Backend confiável:** operações de tabela via **service role** (ignora RLS); auth via chave anon.
- **Rate limit:** 5 tentativas/min por IP em endpoints de auth (Redis ou memória).
- **Dados sensíveis:** sem `console.log` de payloads; tokens apenas em `localStorage`; segredos fora do repositório (`.env` gitignored).
- **Exclusão:** remove do Auth (service role) e da tabela, com guard de integridade.
- **Auditoria:** rastreabilidade de quem/quando (LGPD, dados de saúde).

10. Pendências e evoluções

- **Deploy:** merge do PR #1 em `main`; reativar backend no Render (suspensão); envs de produção; `Site URL` do Supabase para produção.

- **Vercel:** autorização do membro do time para publicar via CLI (quando necessário).
- **Brevo:** remetente "oficial" verificado.
- **Up Visual:** painel provisório com dados mock — integrar com dados reais ou remover.
- **Evoluções:** lembretes automáticos de consulta (agendador), exclusão de pacientes/agentes com regras, telas administrativas, segmentação de auditoria para outras entidades.

11. Referências bibliográficas

1. SUPABASE. *Docs — Auth, Database, Row Level Security, Management API*. <https://supabase.com/docs>.
2. SUPABASE. *supabase-py (Python Client v2)*. <https://supabase.com/docs/reference/python/introduction>.
3. FASTAPI. *FastAPI Documentation — Security, Dependencies, Exception Handling*. <https://fastapi.tiangolo.com>.
4. PYDANTIC. *Pydantic v2 — Data Validation*. <https://docs.pydantic.dev>.
5. REACT / REACT ROUTER. *React Docs; React Router Docs*. <https://react.dev> · <https://reactrouter.com>.
6. VITE / TAILWIND CSS / SHADCN. *Vite; Tailwind CSS v4; shadcn/ui*. <https://vite.dev> · <https://tailwindcss.com> · <https://ui.shadcn.com>.
7. RECHARTS. *Recharts — Charts Library*. <https://recharts.org>.
8. BREVO. *Brevo API v3 — Transactional Email*. <https://developers.brevo.com>.
9. REDIS. *INCR/EXPIRE e padrões de rate limiting*. <https://redis.io>.
10. OWASP. *Authentication Cheat Sheet; Rate Limiting*. <https://owasp.org>.
11. BRASIL. *Lei nº 13.709/2018 (LGPD)*, arts. 16 e 18. <https://www.planalto.gov.br>.
12. IETF. *RFC 3339 — Date and Time (UTC/offsets)*. <https://datatracker.ietf.org/doc/html/rfc3339>.
13. DATE-FNS. *date-fns — formatação de datas*. <https://date-fns.org>.
14. RENDER / VERCEL. *Render Docs; Vercel Docs*. <https://render.com/docs> · <https://vercel.com/docs>.

Documento gerado pela Equipe agenda Saúde — PI2. Dados coletados do repositório e da execução da API em 08/09/2026. Nenhuma credencial é impressa neste documento.